



POLITECNICO DI TORINO

MASTER'S DEGREE IN CYBERSECURITY ENGINEERING

# Design and Implementation of an eBPF-based Security Monitoring Tool

*Supervisor:*

**Prof. Riccardo Sisto**

*Candidate:*

**Simone Romano**

*Company Tutors:*

**Dr. David Soldani**

**Dr. Vincenzo Cosi**

Academic Year 2025-2026

---

This thesis presents the design and implementation of a security monitoring tool that collects and analyzes process-related information directly from the operating system kernel using eBPF (extended Berkeley Packet Filter). By leveraging kernel-level instrumentation, the system captures detailed process events with high accuracy and minimal overhead, overcoming the limitations of traditional user-space monitoring approaches.

The proposed tool adopts a modular and efficient architecture to ensure scalability and real-time data collection. The gathered information is structured to support security analysis, enabling the detection of anomalous behaviors such as unauthorized executions and privilege escalations.

This work demonstrates how kernel-derived process data, combined with eBPF technology, can enhance system visibility and provide a solid foundation for advanced security monitoring and threat detection.

# Acknowledgements

Only in the printed version

# Contents

|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>Acronyms</b>                                                 | <b>1</b> |
| <b>1 Introduction</b>                                           | <b>2</b> |
| 1.1 Context and Motivation . . . . .                            | 2        |
| 1.2 Problem Statement . . . . .                                 | 3        |
| 1.3 Objectives and Thesis Structure . . . . .                   | 5        |
| <b>2 Background and eBPF architecture</b>                       | <b>7</b> |
| 2.1 Linux Kernel Observability . . . . .                        | 7        |
| 2.2 eBPF Fundamentals . . . . .                                 | 11       |
| 2.2.1 Execution Model, Verifier and JIT Compilation . . . . .   | 11       |
| 2.2.2 Hook Selection and Event Semantics . . . . .              | 13       |
| 2.2.3 Maps, Helpers and Kernel-to-Userspace Transport . . . . . | 14       |
| 2.2.4 BTF and CO-RE Portability . . . . .                       | 16       |
| 2.3 Runtime Security and Anomaly Detection . . . . .            | 17       |
| 2.3.1 Point, Contextual and Collective Anomalies . . . . .      | 17       |
| 2.3.2 Policies, Detectors and Alert Generation . . . . .        | 18       |
| 2.3.3 MITRE ATT&CK as a Classification Framework . . . . .      | 19       |
| 2.4 Related Runtime Security Tools . . . . .                    | 20       |
| 2.4.1 Tracee . . . . .                                          | 20       |
| 2.4.1.1 Architecture and Event Pipeline . . . . .               | 20       |

## CONTENTS

---

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| 2.4.1.2  | Unified Event Model . . . . .                           | 22        |
| 2.4.1.3  | Policies and Event Selection . . . . .                  | 22        |
| 2.4.1.4  | Detectors and Derived Events . . . . .                  | 23        |
| 2.4.1.5  | Output and Operational Context . . . . .                | 24        |
| 2.4.1.6  | Portability and Security Boundaries . . . . .           | 24        |
| 2.4.1.7  | Relationship to the Proposed Work . . . . .             | 25        |
| 2.4.2    | Other Relevant eBPF Security Tools . . . . .            | 26        |
| 2.4.3    | Comparative Summary . . . . .                           | 26        |
| 2.5      | Positioning of the Proposed Work . . . . .              | 27        |
| <b>3</b> | <b>Requirements and System Design</b>                   | <b>29</b> |
| 3.1      | Design Goals and Constraints . . . . .                  | 29        |
| 3.1.1    | Target Environment and Compatibility Assumptions .      | 30        |
| 3.1.2    | Functional Requirements . . . . .                       | 31        |
| 3.1.3    | Non-Functional and Operational Requirements . . . .     | 32        |
| 3.2      | System Architecture . . . . .                           | 32        |
| 3.2.1    | Kernel-Space and User-Space Responsibilities . . . . .  | 33        |
| 3.2.2    | End-to-End Event Lifecycle . . . . .                    | 34        |
| 3.3      | Kernel-Space Collection Design . . . . .                | 36        |
| 3.3.1    | Hook Selection and Logical Events . . . . .             | 36        |
| 3.3.2    | Early Event Selection and Kernel-Side Filtering . . . . | 37        |
| 3.4      | Event Contract and Transport Design . . . . .           | 38        |
| 3.4.1    | Stable Binary Event Contract . . . . .                  | 39        |
| 3.4.2    | Perf-Buffer Transport and Failure Boundaries . . . . .  | 40        |
| 3.5      | User-Space Runtime Design . . . . .                     | 41        |
| 3.5.1    | Loading, Attachment and Runtime Lifecycle . . . . .     | 41        |
| 3.5.2    | Decoding, Normalisation and Event Registration . . .    | 42        |
| 3.5.3    | Event Output, Alert Output and Operational Logging      | 43        |
| 3.6      | Policy and Detection Architecture . . . . .             | 44        |

## *CONTENTS*

---

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 3.6.1    | Policy-Based Event Scope . . . . .              | 45        |
| 3.6.2    | Point and Collective Detectors . . . . .        | 46        |
| 3.6.3    | Bounded Local Correlation . . . . .             | 46        |
| 3.6.4    | Alert Evidence and MITRE ATT&CK Metadata . . .  | 47        |
| 3.7      | Design Decisions and Scope Boundaries . . . . . | 48        |
| <b>A</b> | <b>Appendix A</b>                               | <b>49</b> |
| <b>B</b> | <b>Appendix B</b>                               | <b>50</b> |
|          | <b>Bibliography</b>                             | <b>51</b> |

# List of Figures

|     |                                                                                                                                        |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | High-level eBPF execution workflow . . . . .                                                                                           | 12 |
| 2.2 | Complementary evidence exposed by different observation points<br>along a kernel operation. . . . .                                    | 14 |
| 2.3 | Anomaly Detection: Type of Anomalies . . . . .                                                                                         | 17 |
| 2.4 | Tracee high-level architecture . . . . .                                                                                               | 21 |
| 3.1 | Kernel space and userspace architecture . . . . .                                                                                      | 33 |
| 3.2 | Responsibility boundary and end-to-end flow of the proposed<br>monitoring system. . . . .                                              | 35 |
| 3.3 | Conceptual relationship between the binary event record, the<br>user-space event registry, and the resulting normalised event. . . . . | 39 |
| 3.4 | User-space runtime lifecycle and failure boundaries . . . . .                                                                          | 42 |

# List of Tables

|     |                                                                                              |    |
|-----|----------------------------------------------------------------------------------------------|----|
| 2.1 | Observation sources for eBPF runtime monitoring . . . . .                                    | 10 |
| 2.2 | Comparison of eBPF event-transport mechanisms . . . . .                                      | 15 |
| 2.3 | Architectural summary of the runtime security tools considered in this chapter. . . . .      | 27 |
| 3.1 | Functional requirements of the proposed system. . . . .                                      | 31 |
| 3.2 | Non-functional and operational requirements. . . . .                                         | 32 |
| 3.3 | Distinction between physical hooks, producer and support probes, and logical events. . . . . | 37 |

# Acronyms

|              |                                                                |
|--------------|----------------------------------------------------------------|
| eBPF         | extended Berkeley Packet Filter                                |
| CPU          | central processing unit                                        |
| MITRE ATT&CK | MITRE Adversarial Tactics, Techniques, and<br>Common Knowledge |
| LSM          | Linux Security Module                                          |
| JIT          | just-in-time                                                   |
| BTF          | BPF Type Format                                                |
| CO-RE        | Compile Once - Run Everywhere                                  |

# Chapter 1

## Introduction

### 1.1 Context and Motivation

Modern Linux systems execute applications, containers, and infrastructure services whose security-relevant behaviour changes continuously at runtime. Static inspection can describe software before execution, while application logs expose only the information that applications choose to report. Neither source alone provides a complete account of operations such as process creation, credential changes, file access, memory protection changes, namespace activity, or interactions with kernel security mechanisms. Runtime security monitoring therefore requires an observation point that is close enough to the operating system to capture these actions as they occur.

The Linux kernel is a valuable observation layer because it mediates processes, files, credentials, memory, and other shared resources. Kernel-level telemetry can reveal behaviour that is distributed across several applications or omitted from their logs. This visibility is particularly relevant in containerised environments, where workloads remain isolated at the application level but share the host kernel. At the same time, kernel instrumentation is security-sensitive: excessive privileges or unsafe extensions may increase the attack surface, especially across container boundaries [20]. A monitoring design must consequently balance visibility, compatibility, operational cost, and the risks associated with privileged execution.

Runtime security tools increasingly use eBPF to obtain this visibility. Existing systems demonstrate that eBPF can support process, file, network, and security telemetry in cloud-native environments, although their archi-

textures and performance characteristics differ [21]. This thesis focuses on the process and host-security side of that design space.

eBPF provides a controlled runtime inside the Linux kernel. User-space software loads constrained programs that can be attached to supported processes, networking, and security hooks. Before a program is accepted, the kernel verifier analyses its control flow and use of memory, registers, pointers, and helper functions through static analysis. Accepted programs can then collect or transform data when the corresponding hook is reached [18, 22, 23].

This execution model is well suited to runtime monitoring for three reasons. First, collection occurs at the point where the kernel processes the operation, reducing dependence on application-specific instrumentation. Second, eBPF maps and event buffers provide structured communication between kernel and user space. Third, programs can reject irrelevant activity before transporting it to user space, which creates an opportunity to control processing overhead. These properties do not make an eBPF tool automatically safe, portable, or efficient: verifier behaviour, available program types, helper functions, kernel data structures, and attachment mechanisms vary across kernel versions. They instead provide the foundation on which a target-aware monitoring pipeline can be built.

The work was developed in an industrial collaboration and targets Rocky Linux 8.10 with the kernel `4.18.0-553.109.1.el8_10.x86_64`. This environment is important because it combines a long-lived enterprise kernel line with backported functionality. A feature exposed by a newer upstream kernel cannot be assumed to exist, or to behave identically, on this target [28]. Compatibility is therefore treated as an architectural constraint.

Within this target environment, the prototype is designed as a host-level runtime monitor. It collects and processes security-relevant events on the monitored system and performs bounded local detection over individual events and short event sequences.

## 1.2 Problem Statement

The central problem addressed by this thesis is how to convert heterogeneous kernel observations into useful and readable security information on a constrained enterprise Linux target. Capturing an event is only the first step. A practical pipeline must assign a stable identity to the event, transport it

without confusing records produced by different probes, decode typed arguments, apply user-selected scope, and present the result consistently. When these contracts diverge, valid kernel records may be decoded as unknown events, attributed to the wrong event type, or silently discarded.

Individual observations also have different analytical value. Some are relevant in isolation, for example an access to a sensitive file under narrowly defined conditions. Others become relevant only as an ordered group, such as a credential transition followed shortly by process execution or sensitive file access. Following the established anomaly-detection taxonomy, the thesis uses *point event* for a locally significant observation and *short collective event pattern* for a bounded sequence whose combined meaning is more relevant than its elements considered independently [15]. These terms describe deterministic, rule-based detection; they do not imply machine learning, statistical baselines, or unsupervised anomaly detection methods.

Collecting kernel events alone does not make the resulting telemetry immediately useful for security analysis. The proposed system therefore introduces a configurable policy and detection layer that transforms selected runtime observations into focused security information. Policies define the monitoring scope by selecting the event domain relevant to a particular scenario, helping to limit noise and unnecessary processing. Detectors operate on the resulting normalised event stream and encode reusable conditions that can generate an alert from either an individual event or a short, locally correlated sequence. This separation allows collection scope and detection logic to evolve independently, without requiring each analytical change to be implemented inside the eBPF programs. The generated alerts are enriched with metadata based on MITRE ATT&CK [30], providing a standard vocabulary for interpreting the observed behaviour and supporting downstream analysis.

Four constraints shape the problem. First, the target kernel limits which attachment mechanisms, helpers, maps, and portability techniques can be used reliably. Second, the kernel verifier constrains program complexity and memory access, so event enrichment must be divided carefully between kernel and user space. Third, high-frequency hooks can generate enough data to make transport, decoding, rule evaluation, and output formatting more expensive than the probe itself. Fourth, the agent is expected to run beside production workloads; CPU and memory use are therefore evaluation criteria rather than secondary implementation details.

These constraints motivate a selective architecture. The proposed tool uses

eBPF for collection and optional early filtering, while user space owns event normalisation, policy management, detector evaluation, alert generation, and presentation. The user-space runtime is implemented in Go, which was selected in part because libraries such as `libbpfgo` provide Go bindings to `libbpf` for loading eBPF objects and managing programs, maps, and event buffers [3]. The design preserves a stable kernel-to-user-space contract while allowing detection logic to evolve without recompiling every eBPF program.

### 1.3 Objectives and Thesis Structure

The main objective of this thesis is to design, implement, and evaluate an eBPF-based runtime security monitoring prototype that collects process and security-relevant kernel events on the target Rocky Linux environment, normalises them in user space, and evaluates them through configurable policies and local detectors.

The remainder of the thesis develops this objective progressively, moving from the necessary technical foundations to the architecture, implementation, and evaluation of the proposed system:

- **Chapter 2 - Background and eBPF Architecture** introduces Linux runtime instrumentation, the eBPF execution model, and the concepts required to understand process and security monitoring. It also reviews related runtime-security tools and policy-driven detection approaches.
- **Chapter 3 - Requirements and System Design** defines the target environment, functional and operational requirements, architectural components, event flow, and the boundaries between kernel-space collection and user-space processing.
- **Chapter 4 - Implementation** describes the eBPF programs, probe management, binary event contract, Go runtime, decoder, policy and detector subsystems, MITRE ATT&CK metadata, output handling, logging, and container packaging.
- **Chapter 5 - Experimental Evaluation** evaluates functional behaviour, event and detector operation, CPU and memory consumption, and the effects of the implemented filtering and performance controls on the target Rocky Linux environment.

- **Chapter 6 - Conclusions** summarises the results, discusses the principal limitations of the prototype, and identifies possible directions for future development.

## Chapter 2

# Background and eBPF architecture

### 2.1 Linux Kernel Observability

Runtime monitoring requires evidence about operations while they are being executed. Application logs describe domain-specific actions, but their content depends on what each application chooses to record. The Linux kernel offers a complementary observation layer because it mediates process creation, credential changes, file access, memory management, inter-process communication, and other security-relevant operations. Kernel observability is therefore useful when behaviour crosses application boundaries or is not represented consistently in application logs [48, 18].

Linux does not expose all runtime activity through one universal event source. Instead, it provides instrumentation points at different layers of kernel execution. Each mechanism determines when monitoring code runs, which context is available, and how strongly the observation depends on kernel internals.

**System-call observation.** A system call is the controlled interface through which a user-space process requests a service from the kernel. It is not itself an instrumentation mechanism. Monitoring systems commonly observe this boundary through system-call tracepoints, such as entry and exit events. An entry event exposes the requested operation and its original arguments, whereas an exit event exposes the returned result. Correlating the two can

distinguish an attempted operation from one that completed successfully. This view is broad and process-oriented, but it remains close to the user-space request: one system call may traverse several internal functions, and its original arguments may not identify the final kernel object or security decision [51, 48].

**Tracepoints and raw tracepoints.** A tracepoint is a static observation point deliberately inserted into the kernel source by kernel developers. It has a name, belongs to an event category, and defines the data made available when that point is reached. A tracepoint remains inactive until a tracing component is attached to it. In an eBPF-based monitor, this component is an eBPF program loaded from user space and attached to the selected tracepoint. Each time kernel execution reaches that point, the kernel invokes the attached program and provides a tracepoint-specific context containing the declared event fields.

A regular tracepoint gives the eBPF program a structured event context whose layout follows the format exported for that tracepoint. A raw tracepoint refers to the same underlying kernel instrumentation site, but exposes the original arguments passed by the kernel to that site with less intermediate transformation. This reduces preparation performed before the eBPF program runs, while requiring the program to know how each raw argument must be interpreted. Tracepoints are therefore explicit and comparatively stable observation points, whereas raw tracepoints trade some of that abstraction for more direct access to the underlying arguments. Both mechanisms are limited to the tracepoints actually provided by the target kernel [51, 50].

**Kprobes and kretprobes.** A kprobe is a dynamic probe placed at a selected kernel instruction at run time. Monitoring tools usually identify that instruction through the symbol of a kernel function and place the probe at its entry. Unlike a tracepoint, the observation point does not need to have been prepared explicitly by the kernel developer. When execution reaches the probed instruction, the kernel temporarily transfers control to the probing infrastructure, which invokes the attached eBPF program before normal execution continues. The program receives a snapshot of the processor-register state. Function arguments are not presented as a tracepoint-specific event structure; they must be recovered from the appropriate registers according to the architecture and calling convention.

A kretprobe observes the corresponding function-return path. It causes an

attached program to run when an invocation of the probed function finishes and makes the return value available through the register context. A kprobe can therefore describe the operation requested at function entry, while a kretprobe can reveal the result produced at function exit. Entry and return records are separate observations: when both the original arguments and the result are required in one logical event, the monitoring system must save the entry data and correlate it with the return observation, commonly by using the current thread identity.

Kprobes and kretprobes provide access to functions for which no tracepoint exists, but this flexibility increases coupling to kernel implementation details. A renamed or inlined function, a changed signature, a different calling convention, or an unavailable symbol can invalidate the attachment or the argument-decoding logic. They are therefore powerful dynamic mechanisms, but generally require more target-specific validation than tracepoints [45].

**Linux Security Module hooks.** The LSM framework inserts security mediation points into kernel operations involving files, credentials, processes, sockets, and other protected objects. At these points, registered security modules can inspect an operation and, for hooks with authorisation semantics, permit or reject it. Observing an LSM-related function can therefore expose a resolved kernel object and the security-relevant stage of an operation, rather than only the original system-call request. Two concepts should not be confused: an LSM hook is the kernel mediation point, while an eBPF LSM program is one possible program type for attaching logic to supported hooks. On kernels where that program type is unavailable or unsuitable, monitoring tools may instead observe the corresponding security function through a kprobe. An entry-side observation still does not necessarily reveal the final outcome; when the result matters, it must be obtained from a compatible return-side observation [47, 50, 45].

| Observation source     | Information exposed                                       | Main trade-off                                                                                      |
|------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| System-call tracepoint | Arguments at entry or return value at exit.               | Broad process visibility, but limited insight into internal kernel processing and resolved objects. |
| Kernel tracepoint      | Structured fields defined for a specific kernel event.    | Clear semantics and simple decoding, but only where a suitable tracepoint is available.             |
| Raw tracepoint         | Original tracepoint arguments with minimal preprocessing. | More direct access, but requires kernel-aware argument decoding.                                    |
| Kprobe / kretprobe     | Register state at function entry or return.               | Flexible placement, but stronger dependence on kernel symbols, signatures and calling conventions.  |
| LSM-related hook       | Security context, resolved objects and mediation stage.   | Rich security semantics, but attachment and result visibility depend on target-kernel support.      |

Table 2.1: Comparison of the principal observation sources relevant to eBPF-based runtime monitoring [51, 45, 47, 50].

**The role of eBPF.** eBPF is the execution facility attached to an observation point, not a replacement name for the observation point itself. The selected hook causes an eBPF program to run and determines the context received by that program. The program can then extract selected fields, apply an early filter, update maps, or submit an event to user space. Consequently, “tracepoint”, “kprobe”, “LSM hook”, and “eBPF program” describe different parts of the same monitoring path [22, 50].

These mechanisms are complementary. System-call describe requests and results at the user–kernel boundary; general tracepoints expose kernel-defined events; kprobes and kretprobes provide dynamic access to function entry and return; and LSM-related hooks expose security mediation points. A runtime monitor combines them according to the required semantics, available target-kernel features, and acceptable coupling to kernel internals. The engineering objective is to select observation points that provide sufficient and reliable evidence for the intended security analysis [48, 21].

## **2.2 eBPF Fundamentals**

eBPF is a modern technology that makes the Linux kernel programmable at runtime. It allows small programs to be loaded from user space and executed inside the kernel when a selected event occurs. Before execution, the kernel verifies that each program respects the safety constraints associated with its program type and intended attachment point [31, 22].

This approach makes it possible to extend kernel behaviour without modifying the kernel source or developing a traditional kernel module for each new functionality. eBPF programs can be loaded, replaced, and specialised for different purposes, including system observability, performance analysis, network processing, and security. Its principal benefit is therefore controlled programmability: functionality can be introduced close to the kernel operations of interest while remaining subject to verification and well-defined execution constraints [18, 23].

### **2.2.1 Execution Model, Verifier and JIT Compilation**

An eBPF program is compiled into instructions for the BPF virtual machine and submitted to the kernel through the BPF user-space interface. The instruction set defines general-purpose registers, a bounded stack, arithmetic and branch instructions, memory operations, and calls to kernel-provided functions. The program does not execute as unrestricted kernel code: its available context, memory accesses, and callable functions depend on its program type [31, 24].

Because the programs described above are loaded from user space but execute inside the kernel, they cannot be allowed to run without first establishing that they respect the kernel’s safety constraints. An unsafe memory access or an invalid operation in this context could affect the stability and security of the entire system. Linux addresses this requirement through the eBPF verifier, the kernel component responsible for deciding whether a submitted program can be accepted for execution. Rather than trusting the program, the verifier analyses it during the loading process and before permitting its execution; a program that does not satisfy the required safety conditions is rejected and never executed.

The verifier performs a path-sensitive static analysis of the program’s control flow. It follows the possible execution paths while tracking the state of registers and stack locations, distinguishing scalar values from different

classes of pointers, checking memory bounds and alignment, and validating the arguments supplied to helper functions. Paths that may access invalid memory, use uninitialised values, or leave kernel-managed resources unreleased are rejected. To avoid analysing equivalent situations repeatedly, the verifier uses state pruning: when a previously accepted verifier state already covers a new state reached at the same instruction, the redundant path does not need to be explored again [23, 19].

Verification is a necessary safety gate, not a proof that a program implements the intended monitoring semantics. A verifier-accepted program may still read the wrong field, emit an incomplete event, or encode data inconsistently with its user-space decoder. Conversely, semantically reasonable source code may be rejected because the verifier cannot establish the required safety properties for a particular kernel. Verifier behaviour is therefore both a security mechanism and a practical compatibility constraint [23, 37].

Once the verifier accepts a program, the kernel may pass its eBPF instructions to an architecture-specific JIT compiler. The compiler translates the verified eBPF bytecode into native machine instructions for the host architecture. The resulting native code can then be executed directly whenever the program is invoked, avoiding the overhead of processing the eBPF instructions individually at each execution [43, 44].

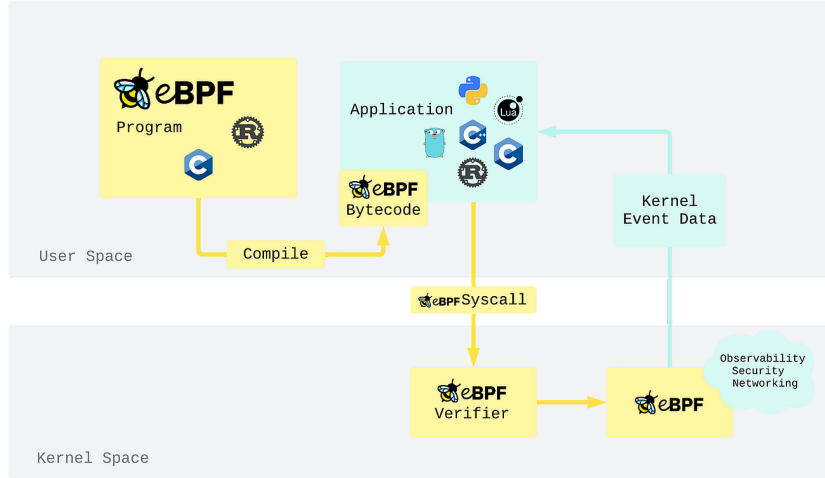


Figure 2.1: High-level eBPF execution workflow.

JIT compilation does not alter the functionality authorised by the verifier.

The translation is performed only after verification and preserves the semantics of the accepted eBPF program. Its availability depends on the host architecture, the kernel build configuration, and the runtime JIT settings.

This completes the basic eBPF execution path considered here. A program is compiled into eBPF instructions, submitted to the kernel, analysed by the verifier and, if accepted, translated into native machine code and executed whenever its associated hook or execution context is invoked [18].

### **2.2.2 Hook Selection and Event Semantics**

An eBPF program type defines the execution contract enforced by the kernel, including the expected context, available helpers, and permitted return semantics. The hook instead identifies the event or kernel execution point that triggers the program, while the attachment mechanism binds the loaded program to a compatible hook. These concepts are related, but they are not interchangeable [50, 24].

The selected observation point determines not only when an eBPF program runs, but also the meaning of the resulting event. A system-call entry hook captures the operation requested by a process and its original arguments. A later tracepoint, dynamic probe, or LSM-related hook may expose kernel-resolved objects or the security mediation stage of the operation. An exit or return-side observation can instead reveal whether the operation completed successfully [51, 45, 47].

These observations represent different stages of the same kernel execution path and should therefore be considered complementary rather than interchangeable. No single hook necessarily exposes the original request, the resolved kernel context, and the final result. A runtime monitor may consequently combine multiple event sources and correlate their records through process identity and temporal proximity.

Hook selection also introduces a compatibility trade-off. Explicit tracepoints provide structured interfaces when the target kernel exports a suitable event, whereas dynamic probes offer broader placement flexibility but depend more strongly on kernel symbols, function signatures, and calling conventions. The appropriate observation point must therefore balance the required security semantics with the capabilities and stability of the deployment kernel [51, 45, 28].

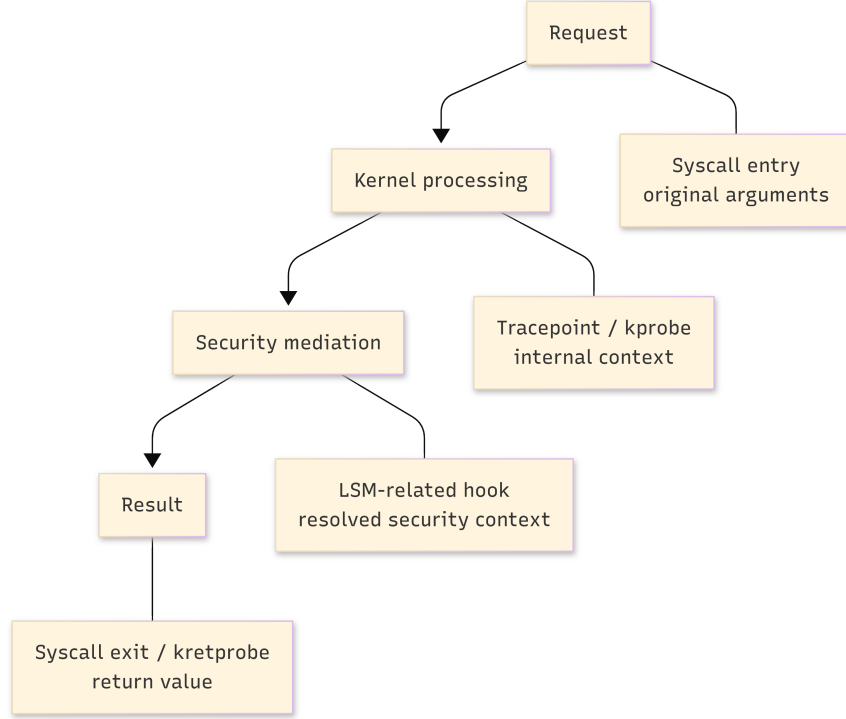


Figure 2.2: Complementary evidence exposed by different observation points along a kernel operation.

### 2.2.3 Maps, Helpers and Kernel-to-Userspace Transport

The execution of an eBPF program is intentionally constrained and usually short-lived. Runtime monitoring, however, requires information to persist beyond a single invocation and eventually reach user space, where more complex processing can be performed. Linux supports this interaction through kernel-managed state and a controlled communication path between eBPF programs and the user-space runtime. Maps, helpers, and event buffers form the main building blocks of this path [39, 24, 49, 40].

Maps are typed kernel objects that allow eBPF programs to preserve state across invocations and exchange data with user space. Each map type defines its own access and concurrency semantics, while its structure and capacity are established at creation time. By moving persistent configuration and temporary state outside the constrained BPF stack, maps extend the

information that a program can retain between executions. Accesses performed by eBPF code remain subject to verifier validation and to the rules imposed by the selected map type [39, 25].

Helpers are kernel-provided functions that extend what an eBPF program can do without granting unrestricted access to kernel internals. They expose selected services through a controlled interface, allowing the kernel to validate how those services are invoked and what data is passed to them. The set of available helpers is determined by the program type and by the capabilities of the target kernel, so support documented for eBPF in general does not imply that the same helper can be used in every execution context. This constraint is particularly relevant to this work, which targets Rocky Linux 8.10 with kernel 4.18.0-553.109.1.el8\_10.x86\_64: helper selection must therefore be validated against the capabilities of the deployed kernel and the program type used by each probe, rather than inferred solely from current upstream documentation [24, 23].

Event transport carries records produced during eBPF execution to user space, where they can be decoded and processed without extending the work performed inside the kernel. The BPF ring buffer provides a shared multi-producer, single-consumer channel designed to use buffer capacity across CPUs and preserve the order in which records are reserved. Perf-event transport follows a different model: a `BPF_MAP_TYPE_PERF_EVENT_ARRAY` directs records to per-CPU perf ring buffers, which are consumed independently and can report lost events.

Both mechanisms rely on finite buffer capacity. When kernel-side producers generate data faster than user space can consume it, a submission may fail or a record may be lost. Event transport must therefore be treated as part of the monitor’s correctness model rather than as a transparent delivery channel [40, 49, 24].

| Mechanism            | Buffer organisation                                   | Main implication                                                                  |
|----------------------|-------------------------------------------------------|-----------------------------------------------------------------------------------|
| BPF ring buffer      | Shared multi-producer, single-consumer buffer         | Shares capacity across CPUs and preserves reservation order between producers.    |
| Perf-event transport | Per-CPU buffers referenced through a perf-event array | Buffers are consumed independently and provide explicit lost-event notifications. |

Table 2.2: Comparison of eBPF event-transport mechanisms [40, 49].

Regardless of the selected transport mechanism, kernel and user space must share a consistent record format so that the transferred data can be decoded into correctly typed events [18].

#### **2.2.4 BTF and CO-RE Portability**

An eBPF program often interacts with kernel data structures whose layout may change across kernel versions and distribution builds. A program compiled against one specific set of kernel headers may therefore rely on field offsets or type definitions that do not match the target system. Portability requires a way to preserve the meaning of those accesses while adapting them to the types exposed by the deployed kernel. BTF and CO-RE provide the metadata and relocation mechanisms used to address this problem.

BTF represents types and related metadata in a compact format understood by the kernel and BPF tooling. BTF may describe C types and strings and can also carry function, line, and relocation information. When kernel BTF is available, loaders and debugging tools can reason about kernel data structures without depending solely on a set of headers copied from one kernel build [42].

CO-RE uses BTF information to adapt an eBPF program to the kernel on which it is loaded. During compilation, Clang stores relocation records that describe the kernel types and fields accessed by the program. At load time, `libbpf` reads the BTF information of the running kernel and updates the relevant eBPF instructions so that they use the actual layout of those structures. As a result, the same compiled object can continue to work when compatible kernel versions place a field at a different offset, without requiring the program to be recompiled on the monitored system [42, 38, 46].

In this thesis, this mechanism helps the eBPF programs access kernel structures according to the layout exposed by Rocky Linux 8.10 with kernel `4.18.0-553.109.1.el8_10.x86_64`. However, CO-RE only adapts compatible differences in type and data-structure layout. It cannot make a missing hook, helper, program type, or attachment mechanism available, and it does not ensure that the target verifier will accept the program. These capabilities must therefore be checked directly on the deployed kernel. This is especially important for enterprise distributions, where backported features may make the effective kernel capabilities differ from those suggested by the upstream version number alone [37, 28].

## 2.3 Runtime Security and Anomaly Detection

Kernel telemetry becomes useful for runtime security only when observed operations are interpreted in relation to a security condition. Detection systems can assign this meaning in different ways. Signature-based detection matches observed activity against predefined patterns, whereas anomaly-based detection identifies deviations from an expected model of behaviour [29, 15].

The detector layer developed in this thesis follows a deterministic, rule-based approach. It evaluates explicit conditions over individual events or short ordered event sequences and does not learn a behavioural baseline or calculate statistical anomaly scores. The point and collective taxonomy introduced below is therefore used to describe the structure of the observations involved in a detection, rather than the algorithm through which the system discovers anomalous behaviour.

### 2.3.1 Point, Contextual and Collective Anomalies

Chandola, Banerjee, and Kumar distinguish point, contextual, and collective anomalies. A point anomaly is an individual observation that is anomalous with respect to the remaining data. A contextual anomaly is anomalous only under particular contextual attributes, such as time or location. A collective anomaly is a related group of observations whose joint behaviour is anomalous, even when the individual observations may not be unusual [15].

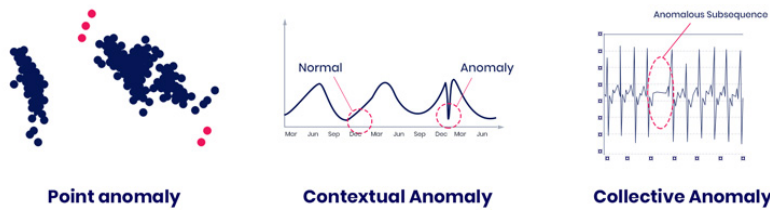


Figure 2.3: Anomaly Detection, Type of Anomalies.[17]

This taxonomy is useful for kernel telemetry. A single operation can be security-relevant when its own attributes satisfy a sufficiently precise rule.

Other behaviours are better represented as an ordered group, for example a credential transition followed shortly by execution or access to a sensitive resource. Event-stream research formalises related ideas through ordered patterns, selection strategies, and bounded temporal constraints [16, 1].

Contextual detection depends on understanding the circumstances in which an event occurs. A monitor running on a single host can interpret an observation using the process information available at that moment, but its view remains limited to the local system. It cannot reliably determine whether the same behaviour is unusual across a wider deployment or inconsistent with organisational expectations that are not visible to the agent. For this reason, the tool developed in this work focuses on deterministic detections over individual events and short, locally correlated sequences. It does not attempt to provide contextual anomaly detection or general-purpose complex event processing.

### **2.3.2 Policies, Detectors and Alert Generation**

Kernel telemetry becomes useful for security analysis only after the collected observations have been placed within a defined monitoring context and evaluated against explicit conditions. The tool developed in this work follows this principle by separating the selection of relevant activity from its security interpretation. Policies define the monitoring scope by selecting the events and process contexts admitted to the analysis pipeline, while detectors operate on the resulting normalised stream and evaluate explicit security conditions. Keeping these responsibilities separate allows the monitoring scope and the detection logic to evolve independently, without moving higher-level analysis into the eBPF programs. Comparable separations between event selection and rule evaluation appear in established runtime security systems [12, 7, 36].

A single-event detector evaluates one observation and emits an alert when its predicates match. A collective detector instead retains the evidence needed to determine whether later observations complete a predefined ordered pattern. Each collective detector defines the local relationship that must hold among the events involved. This allows a pattern to remain associated even when its events originate from different processes, provided that they share the identity required by the detector. A partial match is retained only for a bounded interval and is discarded if the sequence is not completed within that time. These constraints reduce accidental associations between unre-

lated observations, although a completed pattern does not by itself prove malicious intent [16, 1].

An alert is a higher-level result derived from one or more source events. It should identify the detector that matched and preserve sufficient evidence to explain why its condition was satisfied. When the detection concerns a collective pattern, the alert also retains the ordered sequence of observations that completed it. Keeping alerts distinct from raw telemetry allows the two outputs to be retained and routed independently according to operational needs [29, 34].

### **2.3.3 MITRE ATT&CK as a Classification Framework**

The alerts described in the previous subsection preserve the observations that caused a detector to match. However, the meaning of those observations may remain closely tied to the terminology of the local rule that produced the alert. Relating the result to a shared security vocabulary makes it easier to interpret the detected behaviour and communicate it beyond the monitoring tool itself.

MITRE ATT&CK provides such a vocabulary by organising observed adversary behaviour into tactics and techniques. Tactics represent the objectives pursued by an adversary, while techniques describe how those objectives may be achieved. This structure allows detections and security reports produced by different systems to refer to recognised behavioural categories [30, 26].

The tool developed in this work associates ATT&CK metadata with detector definitions and propagates it to the resulting alerts. This connects the low-level runtime evidence preserved by an alert with a broader description of its possible security relevance. Stable ATT&CK identifiers also allow alerts to be interpreted and organised consistently by analysts or downstream systems. The mapping is therefore treated as part of the semantic description of a detector rather than as an additional presentation field [30, 26].

MITRE ATT&CK remains a classification framework: the same technique may appear through different runtime behaviours, while a single observation may support more than one interpretation. A mapping must consequently be reviewed against the specific behaviour represented by the detector. Associating an alert with a technique does not by itself validate the detector or demonstrate complete coverage of that technique [30, 27].

## **2.4 Related Runtime Security Tools**

Existing eBPF-based runtime security tools share the objective of interpreting kernel-derived activity, but they place event selection, detection, and response at different points of the monitoring pipeline. Tracee, Falco, and Tetragon are considered here because they represent three relevant design approaches for positioning the proposed system. Tracee provides the closest architectural reference through its separation of event collection, policies, detectors, and output. Falco centres its design on declarative rules and alert generation, while Tetragon moves filtering and optional response closer to the monitored kernel hook [21].

### **2.4.1 Tracee**

Among the systems considered in this section, Tracee is the closest reference for the tool developed in this work. Both systems separate kernel-level event collection from the analysis performed in user space. This shared structure, rather than a direct feature-by-feature correspondence, makes Tracee the main architectural reference for the proposed system. [8, 9, 21].

#### **2.4.1.1 Architecture and Event Pipeline**

The official architecture overview reproduced in Figure 2.4 presents Tracee as a sequence of connected stages. eBPF programs attached to kernel and networking hooks generate runtime events, which are transferred to user space for collection and enrichment. The resulting observations are then evaluated by the detection layer and made available to external consumers. The diagram therefore illustrates an important architectural property of Tracee: kernel instrumentation is only the first stage of a broader processing pipeline [4].

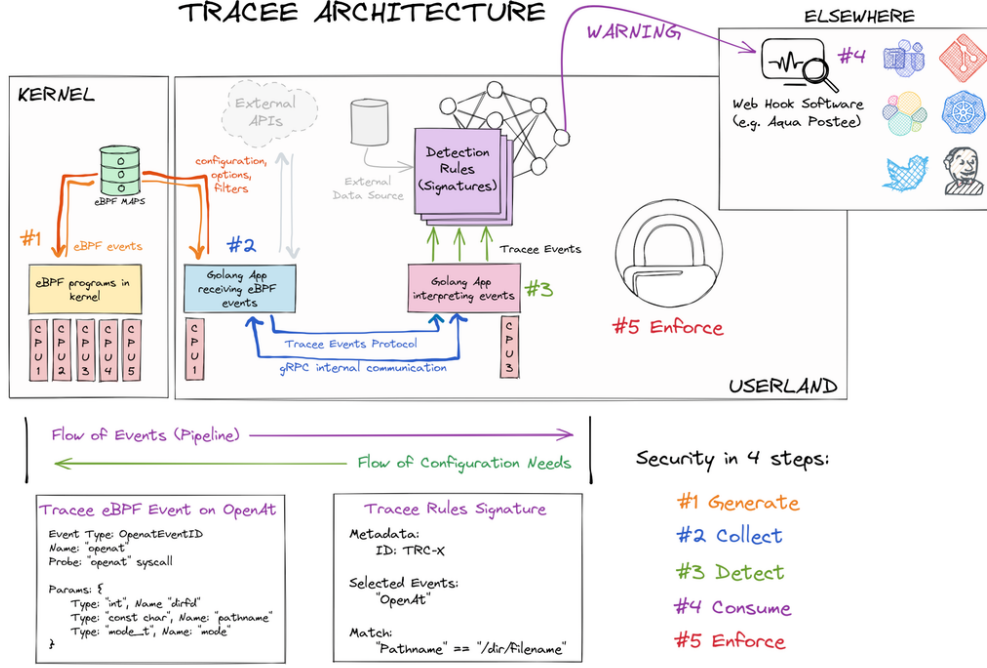


Figure 2.4: Tracee high-level architecture [4].

The terminology used in the figure reflects the Tracee version for which it was published. In particular, the detection stage is represented through the signature-based model used by that release. The current development documentation retains the same broad separation between collection, interpretation, and output, but identifies detectors as the preferred mechanism for implementing new threat detections and derived events. The figure should therefore be read as a high-level architectural overview. [9, 5, 7].

This evolution does not alter the principal architectural idea illustrated by the diagram. Runtime observations originate from different kernel sources but are converted into events before higher-level processing takes place. Detection logic operates on this event representation rather than interacting directly with each underlying probe, while later components determine how the resulting events are enriched, selected, and delivered.

#### **2.4.1.2 Unified Event Model**

The processing stages shown in Figure 2.4 are connected through Tracee’s event abstraction. Observations may originate from system calls, process lifecycle hooks, file and network activity, Linux Security Module instrumentation, or container-related sources, but they are presented to the remainder of the system through a common representation. Policies, detectors, and output components can consequently operate on events without having to manage the attachment semantics of each underlying kernel source [10, 9].

The same abstraction also represents information derived after the original kernel observation has reached user space. Security detections and other derived results become events of their own and can participate in the same processing infrastructure. This reduces the distinction between raw telemetry and higher-level results at the pipeline level, while preserving their different semantic roles [9, 7, 6].

Events can be selected individually or through sets that group related event types. The event catalogue therefore acts both as the interface exposed to users and as the vocabulary shared by the principal user-space components [10].

#### **2.4.1.3 Policies and Event Selection**

Tracee policies determine which part of the available event domain is relevant to a monitoring scenario. A policy combines a scope, which identifies the workloads or execution contexts to which it applies, with rules that select events and optionally constrain their data. The same policy model can be expressed through a plain YAML or JSON document or through a Kubernetes custom resource, allowing the monitoring intent to remain consistent across local and orchestrated deployments [12].

This abstraction separates the decision to observe an event from the logic that interprets its security significance. Policies can restrict collection to the processes, containers, or event families of interest without embedding every selection rule inside a detector. Where the necessary information is available during kernel-side processing, filtering can prevent irrelevant records from entering the user-space pipeline. Conditions that require richer context can instead be completed after the event has been collected and normalised [12].

In the current development model, policy rules can refer not only to ordinary runtime events but also to detectors through their threat metadata. A policy

may therefore select detections according to properties such as severity or MITRE ATT&CK classification without naming every detector individually. This selection is resolved when Tracee starts, so detectors that do not satisfy the configured criteria do not need to participate in runtime dispatch [13].

The policy layer consequently provides more than a final output filter. It defines the portion of the event and detector domain that Tracee prepares for a particular monitoring objective, while leaving the interpretation of matching observations to the detector layer.

#### **2.4.1.4 Detectors and Derived Events**

The signature block shown in Figure 2.4 represents the detection model used when that architecture was documented. Tracee’s current development version introduces detectors as the preferred mechanism for new threat detections and derived events, while retaining the earlier signature system for backward compatibility [5].

A detector declares the input events required by its logic and receives matching observations from the event pipeline. It can then evaluate those observations and produce a derived event when its condition is satisfied. Detectors may be implemented in Go when programmatic control and access to runtime state are required, or described through YAML when the desired condition can be expressed declaratively [7, 6].

Because detector results are represented as events, they are not restricted to a terminal notification channel. They can be selected by policies, routed through the output infrastructure, or consumed by other detectors. Provenance information can preserve the relationship between a derived result and the observations from which it originated [6].

The detector API permits implementations to consume several event types and retain internal state. This makes stateful detection possible, but it should not be interpreted as a predefined general-purpose temporal language. The API provides the mechanisms through which a detector may combine evidence, while the meaning and limits of that correlation remain part of the detector implementation.

#### **2.4.1.5 Output and Operational Context**

Once events have passed through collection, selection, and detection, Tracee’s output subsystem determines how they are presented and delivered. Output configuration is independent from the detector that produced an event. The same observation can therefore be rendered for interactive inspection or serialised for machine processing without changing its detection logic [11].

Tracee supports human-readable tables, structured JSON records, and custom templates. Events can be written to local destinations, forwarded to external logging infrastructure, or delivered through webhooks. Output streams provide an additional routing layer by allowing different portions of the event domain to be sent to independent destinations [11].

The event stream may also be enriched with information maintained outside the original kernel record. Process ancestry and container context can provide the operator with a clearer account of the workload responsible for an observation, while provenance information connects a derived detection to the event from which it originated. This enrichment illustrates why Tracee keeps event collection distinct from the later interpretation and presentation stages: the eBPF program captures the information available at the hook, while user space can relate that record to broader runtime state [6, 11].

Tracee additionally includes forensic capabilities that extend beyond alert generation. Runtime events can guide the collection of artefacts for later investigation, giving the system a broader operational scope than a monitor whose output is limited to telemetry and detector alerts [9].

#### **2.4.1.6 Portability and Security Boundaries**

Tracee’s breadth depends on capabilities exposed by the monitored Linux kernel. Its current installation guidance identifies BTF information and kernel-symbol availability as important prerequisites and explicitly recognises the enterprise Linux 4.18 kernel used by Red Hat Enterprise Linux 8 as a supported exception to its general upstream-version guidance. Individual events may nevertheless depend on additional kernel configuration or attachment support, so the presence of the minimum kernel version does not imply that every Tracee capability is available [2].

This compatibility model is particularly relevant to the target considered in this thesis. It demonstrates that a long-lived enterprise kernel cannot be evaluated solely through its nominal upstream version. BTF availabil-

ity, vendor backports, accessible symbols, verifier behaviour, and supported hooks must be examined on the deployed system.

Tracee also states an explicit trust assumption: the kernel is expected to be trustworthy when monitoring begins. Once a kernel-level adversary controls the system, it may interfere with eBPF execution, suppress observations, or disable the monitoring infrastructure. Tracee can attempt to identify known forms of kernel manipulation after startup, but such detection remains best effort rather than a guarantee against a compromised kernel [14].

This boundary is important when interpreting kernel-derived telemetry. Running the collection programs inside the kernel protects them from ordinary application behaviour, but it does not make the monitoring system independent from the integrity of the kernel on which those programs execute.

#### **2.4.1.7 Relationship to the Proposed Work**

Tracee influenced the proposed system primarily through its separation of event collection, policy-based selection, detector evaluation, and output. The tool developed in this work adopts the same general division of responsibilities: heterogeneous kernel observations are converted into a normalised user-space representation, policies define the monitored event domain, and detectors interpret selected observations without placing all security logic inside the eBPF programs.

The relationship is architectural rather than one of functional equivalence. Tracee provides a broad event catalogue, extensive process and workload context, configurable routing, detector chaining, and forensic capabilities. The prototype developed in this thesis instead concentrates on a selected set of process and host-security events on the specified Rocky Linux kernel. Its detector model is deliberately bounded around deterministic conditions over individual events and short locally correlated sequences.

Tracee therefore serves both as a design reference and as a point of comparison for the contribution of this work. It shows that the main architectural choices on which the proposed system relies are already established in runtime security tooling. The contribution of the thesis lies instead in bringing these ideas together within an independently developed prototype tailored to the selected enterprise environment, where its compatibility, internal event contract, collective detection model, and runtime behaviour can be examined directly.

### **2.4.2 Other Relevant eBPF Security Tools**

Falco is a runtime security engine organised around rules evaluated over event sources. Kernel system-call activity can be captured through its driver architecture, while plugins provide additional sources. Falco rules are declarative YAML objects containing a condition, output description, priority, and optional reusable lists, macros, tags, and exceptions. A matching rule produces an alert that can be routed to configured outputs [33, 36, 34].

This model demonstrates the operational value of readable rules, reusable expressions, explicit exceptions, and an alert-oriented interface. Falco also maintains process and container context used to enrich individual events. However, its documented rule conditions are evaluated against an incoming event, and separate event sources are processed independently; the core rule language is not presented as a general temporal sequence language across heterogeneous sources [32, 35].

Tetragon combines eBPF-based observability with optional runtime enforcement. Its tracing policies specify kernel hooks, captured arguments or return values, and selectors that can match process, namespace, capability, workload, and argument attributes. Supported actions can include emitting an event, delivering a signal, or overriding a return value where the hook and kernel support that operation [55, 56, 52].

Tetragon therefore moves more selection and potential response close to the hook than an architecture centred on user-space detectors. It preserves process-lifecycle context and emits structured events through its own interfaces, but its documented tracing-policy model is principally hook-centred rather than a general language for arbitrary temporal sequences [53, 55]. Its current installation requirements also illustrate the dependence of advanced eBPF functionality on kernel capabilities and BTF availability [54].

### **2.4.3 Comparative Summary**

The systems examined in this section represent different approaches to runtime security. Tracee provides the closest architectural reference because it separates kernel-level collection from the analysis performed in user space. Falco instead places declarative rule evaluation and alert generation at the centre of its design, while Tetragon moves a greater part of event selection and optional response closer to the kernel hook.

These approaches are not directly interchangeable, but they provide use-

ful reference points for the proposed system. Tracee illustrates the event-oriented pipeline that most closely influenced its architecture, whereas Falco and Tetragon establish the boundaries represented by an alert-centred rule engine and an enforcement-oriented design. The main distinctions are summarised in Table 2.3.

| Tool     | Core approach                                                               | Role in the comparison                                                |
|----------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Tracee   | Event-oriented pipeline with separate policy, detection, and output stages. | Closest architectural reference for the proposed system. [12, 7]      |
| Falco    | Declarative rules evaluated over incoming events to generate alerts.        | Reference for a mature rule- and alert-centred model. [36, 35]        |
| Tetragon | Hook-centred policies with in-kernel selectors and optional actions.        | Reference for kernel-side filtering and runtime enforcement. [55, 52] |

Table 2.3: Architectural summary of the runtime security tools considered in this chapter.

## 2.5 Positioning of the Proposed Work

The proposed work occupies a deliberately bounded part of the Linux runtime security design space. It focuses on process and host-security monitoring on Rocky Linux 8.10 with kernel 4.18.0-553.109.1.el8\_10.x86\_64. Because this environment combines an older upstream kernel base with vendor backports, compatibility cannot be inferred from the version number alone. The available hooks, helpers, data layouts, and verifier behaviour must instead be established on the deployed system [28].

Within this environment, eBPF programs collect selected kernel observations and transfer them to a Go runtime through an explicit event contract. The resulting records are converted into a normalised event stream on which policies and detectors perform separate roles. Policies determine the monitored event domain, while configured YAML detectors evaluate deterministic conditions over individual observations or short ordered patterns. Correlation remains local and bounded, but it can associate activity across processes when the observations share the relationship defined by the detector. Alerts retain the evidence that produced the match and may

carry ATT&CK metadata to express its security meaning through a shared classification vocabulary.

The individual mechanisms used by the prototype have clear precedents in existing runtime security systems. Tracee provides the closest architectural reference through its event-oriented separation of collection, policy selection, detection, and output. Falco demonstrates the maturity of declarative event rules and alert generation, while Tetragon illustrates a more kernel-centred design that also supports enforcement [12, 7, 36, 55].

The contribution of this thesis therefore lies in the independent integration of these ideas within a compact prototype tailored to the selected enterprise kernel. Particular attention is given to the compatibility decisions required by that environment, the contract through which kernel records become typed user-space events, and the use of bounded local state for collective detection. These aspects can be examined directly because both the collection and analysis pipeline were developed as part of the work.

## Chapter 3

# Requirements and System Design

The previous chapter established the technical basis for the work and positioned the proposed system within the runtime security landscape. Building on that foundation, this chapter defines the requirements that guide the system and develops the architecture chosen to satisfy them. The discussion focuses on the constraints imposed by the target environment and on the division of responsibility between kernel space and user space.

The design is guided by a simple principle: work that depends directly on the observed kernel operation should remain small and bounded, whereas higher-level interpretation should take place in user space, where it can evolve without increasing verifier-sensitive kernel logic. An explicit event contract connects the two domains. Once an observation crosses this boundary, policies determine whether it belongs to the monitored scope, while independently configured detectors evaluate individual events or short ordered sequences. These choices are presented here at the architectural level, leaving their concrete implementation to Chapter 4 and their experimental assessment to Chapter 5.

### 3.1 Design Goals and Constraints

The system is designed to provide focused visibility into security-relevant host activity rather than an exhaustive account of kernel execution. Its

event domain therefore centres on process execution and on operations that affect the security state of the monitored host. The architecture must allow this domain to be selected according to the monitoring objective, while presenting the resulting observations through a consistent typed event model. Security interpretation should then operate on that model without requiring the detection logic to be embedded in the eBPF programs.

This design is shaped primarily by the characteristics of the target environment and by the continuous nature of runtime telemetry. Kernel capabilities must be established on the concrete enterprise system, since its reported upstream version does not fully describe the available functionality. At the same time, frequent observations require the system to avoid unnecessary collection and to limit the state retained for ongoing detections. Any loss or failure along the event pipeline must also remain observable, so that missing telemetry is not interpreted as evidence that no relevant activity occurred.

### **3.1.1 Target Environment and Compatibility Assumptions**

The target environment is Rocky Linux 8.10 on x86-64, running the exact kernel build `4.18.0-553.109.1.el8_10.x86_64`. This target belongs to a long-lived enterprise kernel line in which selected fixes and features are backported while the reported upstream base version remains stable. The nominal version therefore cannot be used alone to infer the effective eBPF capabilities of the host; the functionality required by the monitoring system must be confirmed on the deployed kernel [28].

The architecture assumes that the monitored host exposes usable kernel BTF information and grants the privileges required to load and attach the selected programs. As discussed in Chapter 2, BTF and CO-RE can adapt compatible differences in kernel data-structure layout, but they cannot provide missing kernel functionality, bypass permissions, or guarantee verifier acceptance [42, 38, 46, 23].

This target-aware design deliberately bounds the portability claim. The system is designed around the identified Rocky Linux environment and is not assumed to operate unchanged across arbitrary distributions or kernel configurations. Dynamic probes introduce an additional dependence on the availability of kernel symbols and compatible function signatures. Loading and attachment failures must consequently be detected before monitoring begins, rather than allowing the runtime to continue with an incomplete observation domain.

### 3.1.2 Functional Requirements

Table 3.1 defines the functional requirements that guide the system architecture. They describe system-level capabilities rather than individual commands or source-code components.

| ID    | Functional requirement                                                                                                                                                                              |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-01 | Collect a selectable set of process and security-relevant events from the monitored host.                                                                                                           |
| FR-02 | Activate only the kernel observation components required to produce the selected logical events and their dependencies.                                                                             |
| FR-03 | Transfer kernel records to user space through the selected perf-event transport and convert valid records into normalised typed events according to an explicit shared contract.                    |
| FR-04 | Load configurable policies that constrain the monitored event and process scope and may restrict the configured detector catalogue before runtime construction.                                     |
| FR-05 | Construct a separate detector engine from the definitions admitted by the active configuration and policies, and evaluate deterministic conditions over single events or bounded ordered sequences. |
| FR-06 | Generate alerts that preserve the observations responsible for a match and propagate the available MITRE ATT&CK metadata.                                                                           |
| FR-07 | Maintain independent presentation paths for raw events, detector alerts, and operational diagnostics.                                                                                               |
| FR-08 | Provide a controlled runtime lifecycle that validates configuration, reports observable failures, responds to cancellation, and releases acquired resources during shutdown.                        |

Table 3.1: Functional requirements of the proposed system.

Together, these requirements define the complete monitoring path. Event selection determines which kernel observations are produced, while the shared contract connects kernel collection to the normalised user-space model. Policies constrain the admitted event domain and may also restrict the detector catalogue during bootstrap. The selected detectors are then constructed within a separate engine and derive evidence-preserving alerts from the resulting event stream. The remaining requirements ensure that telemetry, security results, and operational failures retain distinct meanings throughout the runtime lifecycle.

### 3.1.3 Non-Functional and Operational Requirements

The requirements in Table 3.2 define the compatibility, reliability, and resource constraints under which the functional pipeline must operate.

| ID     | Non-functional or operational requirement                                                                                                                                 |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Operate on the identified enterprise-kernel target with explicit compatibility assumptions and validation against the deployed system.                                    |
| NFR-02 | Keep kernel-side execution and data extraction bounded and acceptable to the target verifier.                                                                             |
| NFR-03 | Maintain a consistent kernel-to-user-space event contract and visibly reject unknown or malformed records before further processing.                                      |
| NFR-04 | Keep detection deterministic, configuration-driven, and independent from event presentation.                                                                              |
| NFR-05 | Bound incomplete collective-detection state in both retention time and capacity.                                                                                          |
| NFR-06 | Report setup failures, transport loss, decoding errors, and detector failures through dedicated diagnostic channels, separate from monitored events and generated alerts. |
| NFR-07 | Assess whether steady-state user-space processing remains below 5% of one CPU core for explicitly defined representative profiles.                                        |

Table 3.2: Non-functional and operational requirements.

NFR-07 defines an evaluation objective rather than an unconditional performance guarantee. Its assessment therefore depends on the monitored workload, enabled events, selected detectors, output configuration, and measurement method. Chapter 5 defines the representative profiles and procedures used for this evaluation.

## 3.2 System Architecture

The architecture separates a control path from a data path. The control path prepares the runtime by validating the configuration and rule definitions, determining which events and detectors are admitted, and resolving each logical event to the kernel programs required for its observation. The data path starts when one of these programs is triggered and carries the resulting observation through transport and user-space processing until it is presented as an event or, when a detector condition is satisfied, as an alert. Operational

diagnostics accompany both paths but remain separate from the monitoring data they describe.

### 3.2.1 Kernel-Space and User-Space Responsibilities

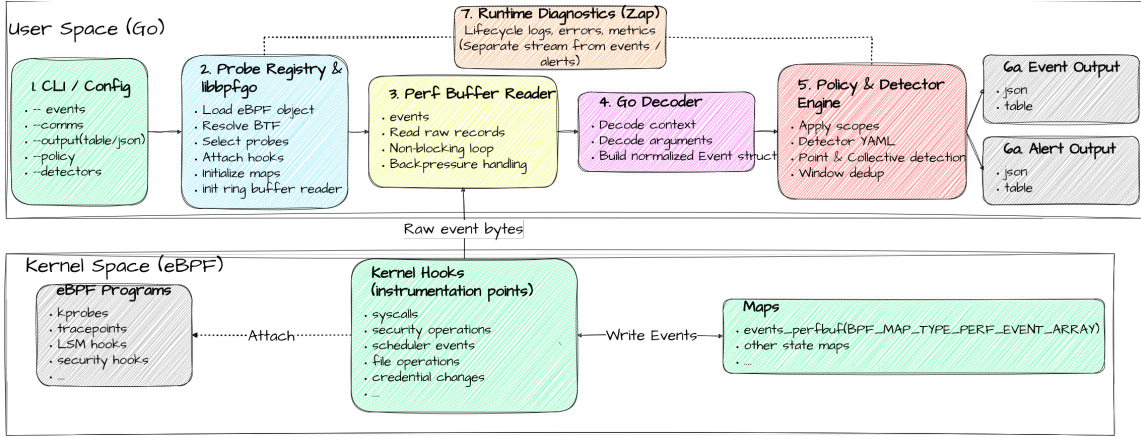


Figure 3.1: Kernel-space and user-space responsibilities.

Figure 3.1 summarises the division of responsibilities between kernel space and user space, together with the control, monitoring, and diagnostic interactions that cross this boundary.

Kernel space is responsible for observing the operations selected during bootstrap and for constructing the corresponding logical events. When an attached hook is reached, the associated eBPF program extracts bounded process context and event-specific arguments, applies the optional exact-match effective-UID guard, and serialises the resulting record according to the shared binary contract. The completed record is then submitted through the perf buffer used as the primary kernel-to-user-space transport channel [24].

Some operations require information that is not available at a single instrumentation point. In these cases, an entry-side program preserves a limited amount of temporary state and a later return-side program completes the event with the operation outcome. This entry/exit model is used only for event construction. It does not introduce policy evaluation or detection logic into the kernel.

User space owns the more changeable and semantically rich responsibilities. During bootstrap, the Go runtime validates the configuration, policies, and detector definitions, derives the effective event scope, resolves the dependencies of the selected logical events, and determines which eBPF programs must be loaded and attached. It manages this lifecycle through `libbpf` and its Go bindings, `libbpfgo` [3]. In this role, `libbpf` provides the user-space facilities required to load eBPF objects and manage programs, maps, and attachments, while `libbpfgo` exposes those facilities to the Go runtime.

During monitoring, the runtime receives binary records from the perf buffer, decodes their context and indexed arguments, and converts them into normalised events. The policy predicates that could not be resolved during bootstrap, including constraints that depend on decoded process information and suppression rules, are then evaluated in user space. Events admitted by this gate may be routed to the event output and are also dispatched to the detector engine constructed from the detector definitions selected during bootstrap. Point detectors evaluate individual events, whereas collective detectors correlate short ordered sequences within bounded temporal and state limits. Only successful detector evaluations produce alerts, which are routed through an output path distinct from raw event presentation.

The diagnostic path is implemented through the structured Zap logging library [57], forming a third independent stream. These diagnostics describe conditions affecting the control or data path while remaining distinct from both monitored events and detector alerts. These records may describe conditions affecting either the control path or the data path, but they are not represented as monitored events or detector alerts.

The boundary is therefore selective rather than absolute. User space controls which kernel programs are loaded and may configure the narrow kernel-side UID guard, while kernel programs may retain short-lived state needed to complete a record. Higher-level policy interpretation, event normalisation, detection, and presentation remain in user space. Conversely, user-space processing depends on the correctness and delivery of the records constructed in kernel space and cannot recover an observation that was rejected before submission or lost during transport.

### **3.2.2 End-to-End Event Lifecycle**

Before collection begins, the configuration, policy rules, and detector definitions are validated. Where the active policies provide exact event allowlists,

these may narrow the event scope requested through the remaining configuration. Policies may also restrict the available detector catalogue before the detector runtime is constructed. The probe registry then resolves the resulting public logical events into the producer programs and internal dependencies required to construct them, while unrelated registered programs are excluded from loading and attachment.

Once monitoring is active, reaching a selected hook invokes its associated eBPF program. The Go runtime decodes the record according to the event registry and converts it into a normalised event. It then evaluates the remaining event and process constraints whose application depends on decoded user-space context. An admitted event may be routed independently to event presentation and to the relevant detectors selected during bootstrap. Disabling event presentation therefore does not prevent detector evaluation. A successful single-event or collective detector evaluation produces an alert that preserves the source observations responsible for the match and propagates the available MITRE ATT&CK metadata.

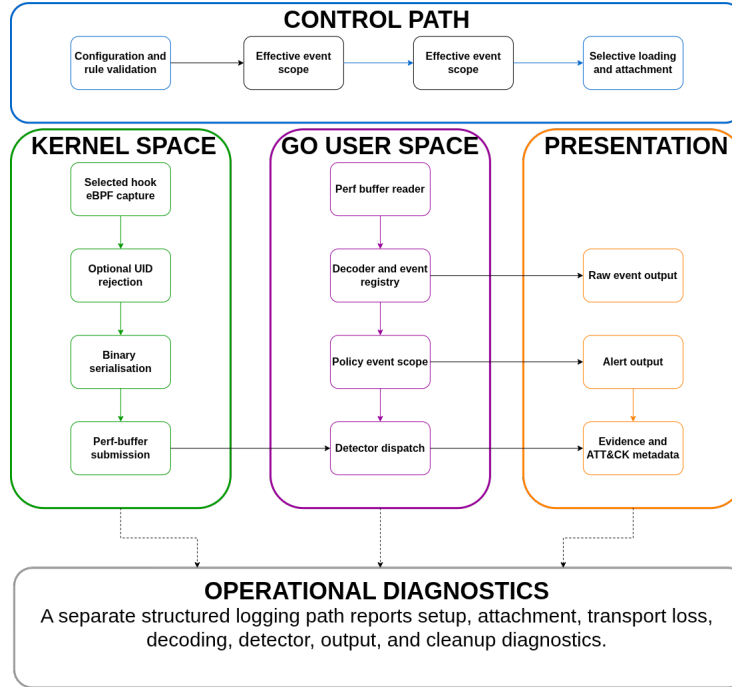


Figure 3.2: Responsibility boundary and end-to-end flow of the proposed monitoring system.

Figure 3.2 summarises the relationship between the control path and the monitoring data path. Operational diagnostics remain separate because they describe conditions affecting these paths rather than representing monitored events or detector alerts.

The diagram does not imply a total ordering of monitored events. Per-event transport uses per-CPU buffering, and collective detectors therefore interpret sequence order according to the order in which records reach the user-space engine. The corresponding transport-loss and ordering boundaries are discussed further in Section 3.4.2.

### **3.3 Kernel-Space Collection Design**

Kernel instrumentation must expose evidence that is both meaningful for the logical event and obtainable on the target kernel. The design therefore does not prescribe one universal attachment mechanism. It combines available observation points while maintaining a stable event abstraction above them.

#### **3.3.1 Hook Selection and Logical Events**

Hook selection begins with the semantics required from the observation. The chosen point must expose the relevant stage of the operation, which may be the initial request, a resolved kernel object, a state transition, or a security mediation decision. An entry-side observation is sufficient when the attempted operation is itself meaningful. When the distinction between success and failure affects the interpretation of the event, a compatible return-side observation is also required.

The choice is further constrained by the observation points available on the concrete target and by their expected cost and robustness. An explicit tracepoint is preferable when it exposes the required semantics, whereas a dynamic probe may extend coverage where no suitable tracepoint exists, at the cost of stronger dependence on kernel symbols, function signatures, and calling conventions. Similarly, a dedicated observation point is preferred over a high-frequency generic hook when it provides equivalent evidence more selectively. These are design-time decisions derived from the properties of the kernel mechanisms discussed in Chapter 2, rather than choices made automatically while the system is running.

Table 3.3 distinguishes the architectural concepts used to describe instru-

mentation and event production.

| Concept        | Architectural meaning                                                                                                                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Physical hook  | A specific point in kernel execution, such as a tracepoint or kernel function, at which an eBPF program can be attached and invoked.                                                                             |
| Producer probe | An eBPF program that directly captures an observation and submits a record for a public logical event. A logical event may be produced by one or more producer probes.                                           |
| Support probe  | An internal eBPF program that does not produce a public event of its own. Instead, it stores, derives, or supplies information required by a producer probe to construct another event correctly.                |
| Logical event  | The stable user-space event type exposed by the monitoring system, identified by a public name and decoding schema. It hides the number and type of physical hooks and probes used to construct the observation. |

Table 3.3: Distinction between physical hooks, producer and support probes, and logical events.

The relationships between these concepts are not necessarily one-to-one. A logical event may require an entry probe to preserve request information and a return probe to complete the record with the operation outcome. Different operation variants may also converge on the same logical event. Conversely, selecting one public event may activate support probes that maintain required context but do not produce independently decodable records. Separating these attachment units from the public event abstraction prevents target-specific probe mechanics from becoming part of the user-facing event contract.

### 3.3.2 Early Event Selection and Kernel-Side Filtering

The public event abstraction introduced in the previous subsection also defines the unit of early selection. Rather than requiring the operator to identify individual eBPF programs, the architecture starts from the logical events that the monitoring scenario is expected to produce. For each selected event, the probe registry determines the producer probes responsible for its construction and any support probes on which those producers depend.

The resulting dependency set determines which registered programs enter the loading lifecycle. Programs that cannot contribute to the selected event domain are excluded before they are submitted to the kernel for verification and loading, and are consequently not attached to their hooks. This approach reduces the active observation surface while preserving the internal probes needed to construct each requested event. It also keeps the public selection interface independent from the particular arrangement of hooks and programs used on the target kernel.

Selective loading applies only to programs represented by the probe registry. It is not a general removal of every unused section contained in the compiled eBPF object, and its correctness depends on the registry describing producers and support dependencies completely. An omitted dependency could leave a selected event without information required for its construction, whereas an incorrectly exposed support probe could make an internal mechanism appear as an independently selectable event.

A further reduction can be applied after an attached program is invoked. The optional exact-UID guard compares the effective UID of the current task with one configured value and rejects non-matching observations before complete context enrichment, record serialisation, and transport. The guard therefore avoids part of the kernel-side and transport work associated with observations known to be outside that narrow scope.

The guard is intentionally limited to a single equality condition and serves as a representative form of inexpensive kernel-side pre-filtering. Additional predicates over values such as process or local cgroup identity could be introduced where they are available reliably and provide a measurable reduction in unnecessary collection. More expressive conditions remain in user space, where they can be evaluated over the complete normalised event without increasing verifier-sensitive kernel logic.

### **3.4 Event Contract and Transport Design**

Kernel-side producers and the user-space consumer require a common representation that remains consistent across heterogeneous event sources. The design combines a bounded binary transport contract with an explicit registry for semantic interpretation. These elements must evolve together: omitting repeated schema metadata from each record reduces transport redundancy, but also means that identifiers, encodings, and registry definitions

must remain aligned.

### 3.4.1 Stable Binary Event Contract

Each public record contains a common event context followed by indexed, typed arguments. The context carries the identity and provenance information shared across event families, including process, host, namespace, and execution context. Event-specific arguments describe the observed operation, while their indices associate the encoded values with the schema maintained by the user-space event registry.

The representation is intentionally not fully self-describing. The kernel producer emits an event identifier and the encoded argument values, whereas the event registry supplies the corresponding public name, argument types, and expected ordering. This avoids transporting the same semantic meta-data with every observation, but creates an explicit compatibility obligation between the kernel definitions, the Go protocol representation, the decoder, and the event registry.

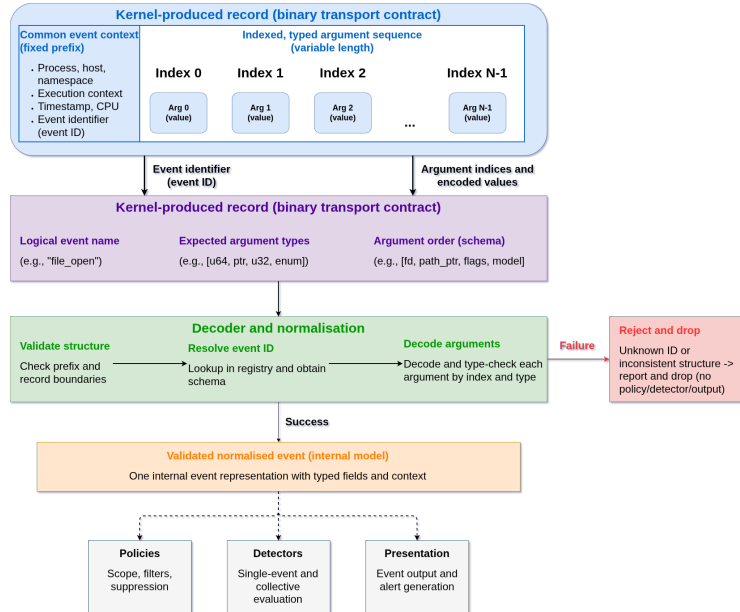


Figure 3.3: Conceptual relationship between the binary event record, the user-space event registry, and the resulting normalised event.

The contract remains valid only while the producer and consumer agree on the common context layout and each public event identifier resolves to a single compatible schema. The indices, types, and encodings of the supplied arguments must also agree with that schema. A record with an unknown identifier or an inconsistent structure is therefore reported and discarded before it reaches policy evaluation, detector processing, or presentation. The runtime does not convert such a record into a partially trusted event.

Decoding and normalisation are related but distinct stages. The decoder first validates the common record structure and resolves the event identifier through the registry. It can then interpret each indexed argument according to the registered schema and verify that the delivered payload is structurally consistent. Only a successfully decoded record is converted into the common normalised representation consumed by policies, detectors, and output components.

The probe registry does not define the binary schema, but it must remain consistent with the same logical interface. Every probe that produces a public event must refer to an event identifier for which the user-space registry provides a valid decoding contract. The implementation mechanisms that realise and preserve these relationships are discussed in Chapter 4.

### **3.4.2 Perf-Buffer Transport and Failure Boundaries**

The operational event path uses perf-event transport, in which records are delivered through per-CPU perf ring buffers ( `BPF_MAP_TYPE_PERF_EVENT_ARRAY`). This mechanism should not be confused with the BPF ring-buffer facility represented by `BPF_MAP_TYPE_RINGBUF`. Although both use circular buffering, the former maintains separate buffers for individual CPUs, whereas the latter provides a shared multi-producer ring buffer [49, 41].

Compatibility with the target kernel was the principal reason for selecting perf-event transport. The perf-event interface is available on the deployed Rocky Linux environment, whereas the BPF ring buffer belongs to a later upstream kernel generation and cannot be assumed to be present through a vendor backport. The transport used by every event was therefore based on the interface verified on the concrete target.

The BPF ring buffer offers advantages on compatible kernels, including shared capacity across CPUs and stronger ordering between records submitted to the same buffer. Perf-event transport instead uses per-CPU buffers,

which may use memory less efficiently under uneven load and do not provide a total ordering across CPUs. These properties were accepted in favour of target compatibility and a simpler operational design.

The choice was also influenced by the evolution of the prototype. An intermediate version maintained both perf-buffer and BPF ring-buffer readers, even though their records eventually entered the same decoding pipeline. Converging on perf-event transport removed duplicated lifecycle and failure-handling paths and ensured that the transport no longer depended on which probe produced the event. This decision does not demonstrate that perf buffers are generally faster, since no controlled comparison between the two mechanisms was performed.

Perf-event delivery remains asynchronous and finite. Records may be lost when a per-CPU buffer cannot be consumed quickly enough, and the resulting loss notification does not allow the missing observations to be reconstructed. A delivered record may instead fail during decoding if it violates the active binary contract. Transport loss and decoding failure are therefore separate conditions, neither of which implies that the corresponding kernel operation did not occur.

The per-CPU organisation also limits ordering guarantees. Collective detectors evaluate sequences according to the order in which records reach the user-space engine, since the architecture does not reconstruct a total causal order across CPUs. The effects of transport pressure, loss, and concurrent delivery are assessed in Chapter 5.

## **3.5 User-Space Runtime Design**

The Go runtime coordinates kernel-facing resources with the higher-level event pipeline. Its components form a controlled runtime lifecycle in which resources are acquired only after validation and remain under clear ownership, while failures affecting individual events are contained without disrupting the main processing loop.

### **3.5.1 Loading, Attachment and Runtime Lifecycle**

The runtime follows an ordered lifecycle that separates setup, active event processing, and cleanup. Figure 3.4 summarises this lifecycle and distinguishes fatal setup failures from event-local failures that do not interrupt

monitoring.

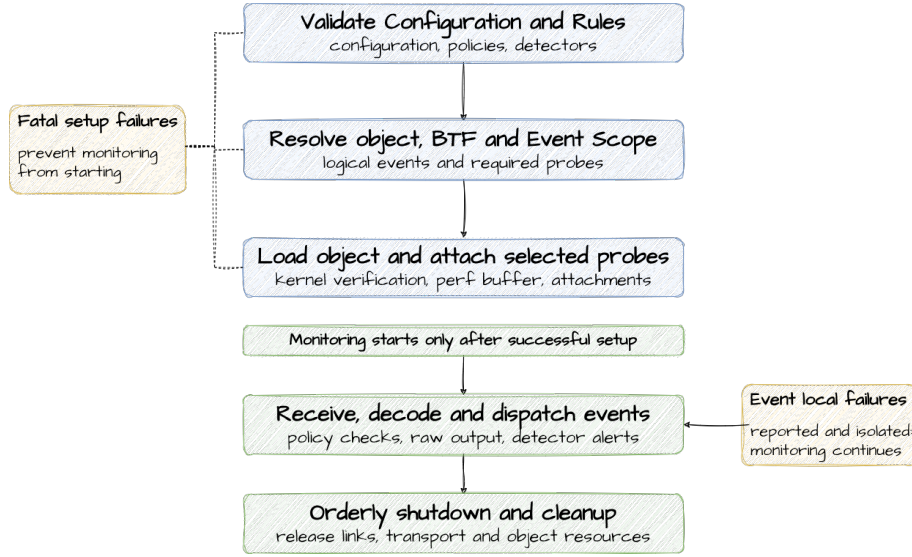


Figure 3.4: Simplified user-space runtime lifecycle.

Startup is ordered so that the configuration and external definitions are accepted before kernel resources are acquired. The runtime normalises and validates the configuration, resolves the eBPF object and BTF source, and then loads and validates the policy and detector definitions. Active policies may narrow the effective event scope and detector catalogue during this bootstrap phase, before the detector runtime is constructed. The selected logical events are subsequently resolved through the probe registry into their producer and support probes, while unrelated registered programs are excluded from autoload.

### 3.5.2 Decoding, Normalisation and Event Registration

The decoder forms the user-space validation boundary for records received from the perf buffer. It first verifies that the common event context and each encountered argument payload can be read within the delivered record. The event identifier is then resolved through the event registry, and argument indexes are interpreted according to the registered schema. This process produces a typed `bufferdecoder.Event` containing the common context,

the logical event name, and the decoded arguments.

Only a record with a decodable context, a known event identifier, and argument records that can be interpreted using the registered schema enters the policy and detector pipeline. The decoder rejects records that cannot be safely reconciled with the registered schema and configured bounds, for example when the payload is truncated, the event identifier is unknown, or an encoded string exceeds the permitted size. It does not, however, require every schema argument to be present, reject duplicate argument indexes, or require the complete payload to be consumed. The decoding contract therefore establishes bounded and typed interpretation rather than strict canonical encoding.

The event registry and the probe registry describe separate aspects of the same logical interface. The event registry is authoritative for public event identifiers, names, and argument schemas used by decoding and configuration validation. The probe registry instead defines the attachable eBPF programs, their kernel hooks and attachment methods, and any support dependencies needed to produce a logical event.

The separation between the two registries preserves distinct architectural responsibilities: the event registry defines the decoding contract, whereas the probe registry defines how the corresponding observations are produced. Their alignment is enforced through consistency checks that verify that public logical events have a valid decoder definition and that internal support probes remain outside the public event-selection interface. The relationships between physical probes and logical events are already discussed in Section 3.3.1.

The decoded event provides the typed representation passed to subsequent user-space stages, while its use by policies, detectors, and presentation components is discussed in Sections 3.6 and 3.5.3. Records that cannot be decoded are instead reported and discarded at this boundary, preventing unknown or partially interpreted data from entering the normalised event stream while allowing later valid records to continue through the monitoring loop.

### **3.5.3 Event Output, Alert Output and Operational Logging**

The output architecture preserves three semantically distinct result classes: monitored events, detector alerts, and operational diagnostics. This separa-

tion is not merely presentational. Event output exposes telemetry admitted by the active policy scope, alert output exposes security results derived from detector matches, and operational logging describes the state of the monitoring infrastructure itself.

Each class follows an independent output contract. Event presentation is a terminal consumer of the normalised event stream and may be disabled without removing the corresponding observations from policy evaluation or detector dispatch. Alert presentation is similarly separated from detector evaluation: changing its representation does not alter the matching condition or the structured evidence retained by the alert. Presentation decisions therefore do not affect collection and detection semantics.

Operational diagnostics remain outside both security-data paths. The runtime uses the Uber Zap library to provide structured logging for lifecycle state, libbpf messages, transport loss, decoding and detector failures, presentation errors, and cleanup conditions. Zap is used only for diagnostic reporting; it does not serialise monitored events or detector alerts. Events and alerts are handled by their respective output components and retain formats suited to interactive or machine-oriented consumption [57].

This separation prevents infrastructure conditions from being interpreted as monitored behaviour. For example, a decoding failure is reported through Zap rather than being converted into an event, while a detector failure is not represented as a security alert. Diagnostic verbosity and encoding can therefore be adapted to the deployment environment without changing the event or alert interfaces exposed to downstream consumers.

The three paths also define separate failure boundaries. A presentation or diagnostic failure must not invalidate a successfully decoded event, alter detector evaluation, or prevent the remaining paths from progressing. Some startup failures occur before the Zap logger is constructed and are returned directly to the process entry point. This is an outer lifecycle boundary rather than an additional output class; once the managed runtime has been created, Zap provides the structured diagnostic path shown in Figures 3.1 and 3.3.

## **3.6 Policy and Detection Architecture**

Policies and detectors are separate, configuration-driven stages over the normalised event stream. This separation follows an established runtime secu-

rity pattern rather than constituting novelty by itself. Its role in the prototype is to keep collection scope, detector selection, security interpretation, and presentation independently configurable within a bounded host-local design.

### **3.6.1 Policy-Based Event Scope**

A policy defines the portion of the monitored domain that is relevant to a particular runtime configuration. Its event rules determine whether a normalised event is admitted according to its identity and selected process attributes. Suppression rules take precedence over non-suppressive matches, while the absence of configured policies leaves the decoded event stream unrestricted by this layer.

Part of this scope can be resolved before monitoring begins. When the active non-suppressive rules provide explicit event allowlists, their combined event domain can narrow the selection passed to the probe registry. An event scope also supplied through the remaining configuration is combined with this result before the required producer and support probes are resolved. Constraints that depend on decoded process information, together with suppression decisions, remain in the user-space path. Policy handling is therefore divided between an optional bootstrap-time reduction of the observation domain and an authoritative admission decision over decoded events; it is not implemented as a complete kernel-side policy engine.

Policies may also restrict the configured detector catalogue during bootstrap. Detector selectors can refer to explicit detector identifiers or to metadata such as severity, MITRE ATT&CK classification, tags, and whether a detector retains state. Values within the same selector category are combined as alternatives, while constraints from different categories must be satisfied together. Exclusions take precedence over inclusions, and the selections produced by multiple non-suppressive policies are combined before the detector engine is constructed.

This detector selection remains separate from event admission. It determines which configured detector definitions are instantiated and therefore avoids allocating runtime or correlation state for excluded detectors, but it does not create distinct detector pipelines for individual policies. Policy mode and intent metadata do not alter the execution model, and matched policy names are not automatically propagated into detector alerts. These boundaries distinguish the implemented scope model from the broader pol-

icy capabilities reviewed in Chapter 2.

### **3.6.2 Point and Collective Detectors**

Detector definitions are loaded independently from policies. Their event references and conditions are validated before the runtime begins, and the dispatcher indexes them by the events they consume. This avoids evaluating every detector for every event and isolates a detector-local error from the remaining detector set.

A point detector evaluates deterministic predicates over one normalised event. When the predicates match, it creates an alert containing that source event. A collective detector instead defines an ordered sequence of event conditions. It retains a partial match until a later admitted event advances the sequence, the sequence completes, or its local state is discarded. Neither model learns a behavioural baseline or calculates a statistical anomaly score.

Detector event declarations are consumer requirements, not attachment instructions. They are checked against the event registry, but they do not automatically enlarge the collection scope. An operator must select compatible events through the monitoring configuration or policies. A valid detector may otherwise remain inactive because one of its required observations is not produced.

Sequence order is the order in which events arrive at the user-space detector engine. The design does not search backwards, reorder observations, or infer a causal graph. This bounded model supports the short collective patterns defined for the prototype while avoiding a claim of general complex-event processing [16, 1].

### **3.6.3 Bounded Local Correlation**

Each collective detector defines how the events in its ordered sequence must be related. The available local relationships distinguish a stable process, an immediate parent-to-child transition, the same file resource, the same local cgroup, or a conjunction of these dimensions. A partial match is retained only within a short configurable interval and a bounded number of correlation keys. When the sequence completes, the alert preserves the ordered events that satisfied it; otherwise, the incomplete state expires, is replaced for the same key, or is evicted.

Stable process correlation distinguishes separate process lifetimes even when the operating system later reuses the same numeric process identifier. Process-tree correlation remains deliberately shallow: a sequence may stay within one process or move from a parent observation to its immediate child, but it does not traverse arbitrary ancestry or construct a persistent process graph.

Resource correlation identifies the same local file object rather than relying on a pathname string. This permits related activity from different processes to be joined while avoiding the assumption that an equal path always denotes the same object. Cgroup correlation can similarly connect processes sharing a local kernel cgroup identity. It does not attribute events to a Kubernetes pod, workload, tenant, or cluster-wide identity.

A composite relationship requires every configured dimension to agree; it is a conjunction rather than a fallback among alternative keys. User-session correlation is rejected because the normalised event model does not expose a stable session identity, and UID alone would merge unrelated activity.

Only one partial sequence is retained for a detector and correlation key. A new first step for that key can replace the existing incomplete sequence. State is bounded by time and by a per-detector key limit, although total memory also depends on detector count, sequence length, and retained event payloads. Expired state is reclaimed during relevant detector activity or an explicit flush, not by a general periodic scheduler in the main runtime.

#### **3.6.4 Alert Evidence and MITRE ATT&CK Metadata**

An alert is a derived detector result rather than an operational log message. A point alert preserves the source event that satisfied its predicates, while a collective alert preserves the ordered sequence accumulated by the detector. This evidence makes the result traceable to the runtime observations on which it is based and allows machine-oriented output to retain more detail than a compact interactive summary.

Detector definitions may associate this evidence with MITRE ATT&CK tactics, techniques, data sources, and data components. The metadata is copied into the resulting alert, providing a shared vocabulary through which the intended security meaning can be communicated and organised [30]. The mapping is part of detector configuration rather than policy selection.

Validation establishes that configured tactic and technique identifiers follow the expected syntactic form. It does not consult an authoritative ATT&CK

catalogue at runtime, verify the semantic correctness of the mapping, or demonstrate complete coverage. These limitations do not remove the value of the metadata: preserving the mapping alongside the evidence supports interpretation, traceability, and later coverage analysis. The final assessment of representative detector mappings belongs to Chapter 5.

### **3.7 Design Decisions and Scope Boundaries**

The architecture follows a series of deliberate trade-offs. Bounded kernel-space collection is preferred to embedding complex, verifier-sensitive analysis in each probe. The Go runtime owns policy and detector semantics so that those definitions can evolve without recompiling the kernel programs. Selective loading and attachment are preferred to activating every registered probe, and an explicit shared event contract is preferred to ad hoc payload interpretation.

Perf buffers are used because they are supported by the reference environment and runtime stack; BPF ring-buffer transport remains a non-implemented alternative. Policy scope is kept separate from detector logic, and detector results are kept separate from operational logging. Collective detection uses short, local, evidence-preserving sequences rather than persistent graphs or a general-purpose event-processing system.

These decisions define the limits of the prototype. It does not implement enforcement, prevention, machine-learning anomaly discovery, user-session correlation, arbitrary process ancestry, cluster-wide correlation, or Kubernetes workload attribution. It does not claim complete Linux event or MITRE ATT&CK coverage, universal kernel portability, or universal compliance with the performance objective.

Container packaging supports later operational use but is an implementation concern described in Chapter 4. A wider platform may eventually aggregate alerts from several monitored hosts and add deployment context, but that external capability is neither implemented nor evaluated as part of this thesis. Within its stated boundary, the proposed system provides a coherent path from selected kernel observations to normalised telemetry and evidence-preserving local detections on the identified enterprise kernel.

## Appendix A

## Appendix A

Appendix text here.

## Appendix B

## Appendix B

Appendix text here.

# Bibliography

- [1] Jagrati Agrawal et al. “Efficient Pattern Matching over Event Streams”. In: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. Ed. by Jason Tsong-Li Wang. New York, NY, USA: Association for Computing Machinery, 2008, pp. 147–160. ISBN: 978-1-60558-102-6. DOI: [10.1145/1376616.1376634](https://doi.org/10.1145/1376616.1376634).
- [2] Aqua Security. *Installing Tracee*. Official Tracee documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/install/> (visited on 07/29/2026).
- [3] Aqua Security. *libbpfgo: An eBPF Library for Go Powered by libbpf*. Official project repository. 2026. URL: <https://github.com/aquasecurity/libbpfgo> (visited on 07/27/2026).
- [4] Aqua Security. *Tracee Architecture*. Official architecture documentation for Tracee version 0.15. 2023. URL: <https://aquasecurity.github.io/tracee/v0.15/contributing/architecture/> (visited on 07/29/2026).
- [5] Aqua Security. *Tracee Custom Events*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/events/custom/overview/> (visited on 07/29/2026).
- [6] Aqua Security. *Tracee Detector API Reference*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/api-reference/> (visited on 07/27/2026).
- [7] Aqua Security. *Tracee Detectors*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/> (visited on 07/27/2026).
- [8] Aqua Security. *Tracee Documentation*. 2026. URL: <https://aquasecurity.github.io/tracee/latest/> (visited on 07/17/2026).

## BIBLIOGRAPHY

---

- [9] Aqua Security. *Tracee Documentation Overview*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/overview/> (visited on 07/29/2026).
- [10] Aqua Security. *Tracee Events*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/events/> (visited on 07/27/2026).
- [11] Aqua Security. *Tracee Output Configuration*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/outputs/> (visited on 07/27/2026).
- [12] Aqua Security. *Tracee Policies: Overview*. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/> (visited on 07/17/2026).
- [13] Aqua Security. *Tracee Policy Rules*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/rules/> (visited on 07/29/2026).
- [14] Aqua Security. *Tracee Security Model*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/security-model/> (visited on 07/29/2026).
- [15] Varun Chandola, Arindam Banerjee, and Vipin Kumar. “Anomaly Detection: A Survey”. In: *ACM Computing Surveys* 41.3 (July 2009), 15:1–15:58. DOI: [10.1145/1541880.1541882](https://doi.org/10.1145/1541880.1541882).
- [16] Gianpaolo Cugola and Alessandro Margara. “Processing Flows of Information: From Data Stream to Complex Event Processing”. In: *ACM Computing Surveys* 44.3 (June 2012), 15:1–15:62. DOI: [10.1145/2187671.2187677](https://doi.org/10.1145/2187671.2187677).
- [17] Einar & Partners Research Unit. *Mystery of Predictions in Anomaly Detection*. Einar & Partners Research Unit. URL: <https://research.einar.partners/mystery-of-predictions-in-anomaly-detection/> (visited on 07/28/2026).
- [18] Bolaji Gbadamosi et al. “The eBPF Runtime in the Linux Kernel”. In: *arXiv preprint arXiv:2410.00026* (2024). DOI: [10.48550/arXiv.2410.00026](https://doi.org/10.48550/arXiv.2410.00026). URL: <https://arxiv.org/abs/2410.00026>.
- [19] Elazar Gershuni et al. “Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions”. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. Ed. by Kathryn S. McKinley and Kathleen Fisher. PLDI ’19. Association for Computing Machinery, 2019, pp. 1069–1084. DOI: [10.1145/3322271.3322300](https://doi.org/10.1145/3322271.3322300).

## BIBLIOGRAPHY

---

- 10.1145/3314221.3314590. URL: <https://doi.org/10.1145/3314221.3314590>.
- [20] Yi He et al. “Cross Container Attacks: The Bewildered eBPF on Clouds”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. Ed. by Joseph A. Calandrino and Carmela Troncoso. Anaheim, CA: USENIX Association, Aug. 2023, pp. 5971–5988. ISBN: 978-1-939133-37-3. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/he>.
- [21] Jin Her et al. “An In-Depth Analysis of eBPF-Based System Security Tools in Cloud-Native Environments”. In: *IEEE Access* 13 (2025), pp. 155588–155604. DOI: 10.1109/ACCESS.2025.3605432. URL: <https://ieeexplore.ieee.org/document/11146725/>.
- [22] Linux Kernel Community. *eBPF Userspace API*. Linux kernel 6.9 documentation. 2024. URL: <https://www.kernel.org/doc/html/v6.9/userspace-api/ebpf/index.html> (visited on 07/17/2026).
- [23] Linux Kernel Community. *eBPF Verifier*. 2026. URL: <https://docs.kernel.org/bpf/verifier.html> (visited on 07/17/2026).
- [24] Linux man-pages project. *bpf-helpers(7) — List of eBPF Helper Functions*. 2026. URL: <https://man7.org/linux/man-pages/man7/bpf-helpers.7.html> (visited on 07/27/2026).
- [25] Linux man-pages project. *bpf(2) — Perform a Command on an Extended BPF Map or Program*. 2026. URL: <https://man7.org/linux/man-pages/man2/bpf.2.html> (visited on 07/27/2026).
- [26] MITRE Corporation. *Get Started with MITRE ATT&CK*. 2026. URL: <https://attack.mitre.org/resources/> (visited on 07/27/2026).
- [27] MITRE Corporation. *MITRE ATT&CK Frequently Asked Questions*. 2026. URL: <https://attack.mitre.org/resources/faq/> (visited on 07/27/2026).
- [28] Red Hat. *What Is Backporting and How Does It Affect Red Hat Enterprise Linux?* Red Hat Customer Portal. June 2025. URL: <https://access.redhat.com/solutions/57665> (visited on 07/27/2026).
- [29] Karen Scarfone and Peter Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. Special Publication 800-94. National Institute of Standards and Technology, Feb. 2007. DOI: 10.6028/NIST.SP.800-94. URL: <https://doi.org/10.6028/NIST.SP.800-94>.

## BIBLIOGRAPHY

---

- [30] Blake E. Strom et al. *MITRE ATT&CK: Design and Philosophy*. Tech. rep. The MITRE Corporation, Mar. 2020. URL: <https://www.mitre.org/news-insights/publication/mitre-attck-design-and-philosophy>.
- [31] Dave Thaler. *BPF Instruction Set Architecture (ISA)*. RFC 9669. Internet Engineering Task Force, Oct. 2024. DOI: [10.17487/RFC9669](https://doi.org/10.17487/RFC9669). URL: <https://www.rfc-editor.org/info/rfc9669>.
- [32] The Falco Project Authors. *Falco Event Sources*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/> (visited on 07/27/2026).
- [33] The Falco Project Authors. *Falco Kernel Event Architecture*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/kernel/architecture/> (visited on 07/27/2026).
- [34] The Falco Project Authors. *Falco Outputs*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/outputs/> (visited on 07/27/2026).
- [35] The Falco Project Authors. *Falco Rule Conditions*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/conditions/> (visited on 07/27/2026).
- [36] The Falco Project Authors. *Falco Rules*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/> (visited on 07/27/2026).
- [37] The Linux Kernel Documentation. *BPF Design Q&A*. 2026. URL: [https://docs.kernel.org/bpf/bpf\\_design\\_QA.html](https://docs.kernel.org/bpf/bpf_design_QA.html) (visited on 07/27/2026).
- [38] The Linux Kernel Documentation. *BPF LLVM Relocations*. 2026. URL: [https://docs.kernel.org/bpf/llvm\\_reloc.html](https://docs.kernel.org/bpf/llvm_reloc.html) (visited on 07/27/2026).
- [39] The Linux Kernel Documentation. *BPF Maps*. 2026. URL: <https://docs.kernel.org/bpf/maps.html> (visited on 07/27/2026).
- [40] The Linux Kernel Documentation. *BPF Ring Buffer*. 2026. URL: <https://docs.kernel.org/bpf/ringbuf.html> (visited on 07/27/2026).
- [41] The Linux Kernel Documentation. *BPF Ring Buffer*. 2026. URL: <https://docs.kernel.org/bpf/ringbuf.html> (visited on 08/03/2026).
- [42] The Linux Kernel Documentation. *BPF Type Format (BTF)*. 2026. URL: <https://docs.kernel.org/bpf/btf.html> (visited on 07/27/2026).

## BIBLIOGRAPHY

---

- [43] The Linux Kernel Documentation. *Classic BPF vs eBPF*. Describes execution through architecture-specific JIT compilation and, when JIT compilation is not used, through the `__bpf_prog_run()` interpreter. URL: [https://docs.kernel.org/bpf/classic\\_vs\\_extended.html](https://docs.kernel.org/bpf/classic_vs_extended.html) (visited on 07/27/2026).
- [44] The Linux Kernel Documentation. *Documentation for /proc/sys/net/: BPF JIT Controls*. 2026. URL: <https://docs.kernel.org/admin-guide/sysctl/net.html#bpf-jit-enable> (visited on 07/27/2026).
- [45] The Linux Kernel Documentation. *Kernel Probes (Kprobes)*. 2026. URL: <https://docs.kernel.org/trace/kprobes.html> (visited on 07/27/2026).
- [46] The Linux Kernel Documentation. *libbpf Overview*. 2026. URL: [https://docs.kernel.org/bpf/libbpf/libbpf\\_overview.html](https://docs.kernel.org/bpf/libbpf/libbpf_overview.html) (visited on 07/27/2026).
- [47] The Linux Kernel Documentation. *Linux Security Module Development*. 2026. URL: <https://docs.kernel.org/security/lsm-development.html> (visited on 07/27/2026).
- [48] The Linux Kernel Documentation. *Linux Tracing Technologies*. 2026. URL: <https://docs.kernel.org/trace/index.html> (visited on 07/27/2026).
- [49] The Linux Kernel Documentation. *Perf Ring Buffer*. 2026. URL: [https://docs.kernel.org/userspace-api/perf\\_ring\\_buffer.html](https://docs.kernel.org/userspace-api/perf_ring_buffer.html) (visited on 07/27/2026).
- [50] The Linux Kernel Documentation. *Program Types and ELF Sections*. 2026. URL: [https://docs.kernel.org/bpf/libbpf/program\\_types.html](https://docs.kernel.org/bpf/libbpf/program_types.html) (visited on 07/27/2026).
- [51] The Linux Kernel Documentation. *Using the Linux Kernel Tracepoints*. 2026. URL: <https://docs.kernel.org/trace/tracepoints.html> (visited on 07/27/2026).
- [52] The Tetragon Authors. *Tetragon Enforcement*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/enforcement/> (visited on 07/27/2026).
- [53] The Tetragon Authors. *Tetragon Events*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/events/> (visited on 07/27/2026).

## BIBLIOGRAPHY

---

- [54] The Tetragon Authors. *Tetragon Installation FAQ*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/installation/faq/> (visited on 07/27/2026).
- [55] The Tetragon Authors. *Tetragon Tracing Policies*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/> (visited on 07/27/2026).
- [56] The Tetragon Authors. *Tetragon Tracing Policy Selectors*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/selectors/> (visited on 07/27/2026).
- [57] Uber Go. *Zap: Structured, Levelled Logging in Go*. URL: <https://github.com/uber-go/zap> (visited on 07/31/2026).

*To my family and friends, for their support.*